

Teambook Runtime Terrors

Agosto 2026

Índice

1. Búsqueda	1
1.1. Búsqueda binaria	1
1.2. Búsqueda ternaria	2
2. Estructuras de datos	2
2.1. Fenwick Tree (BIT)	2
2.2. Segment Tree (min)	2
2.3. Segment Tree con Lazy Propagation (min + rango suma)	2
2.4. Segment Tree Persistente (min point update)	3
2.5. UnionFind (DSU)	4
2.6. Sparse Table (RMQ $O(1)$)	4
2.7. Ordered Set (PBDS)	4
3. Geometria	4
3.1. Graham's Scan	4
4. Grafos	5
4.1. Bellman Ford	5
4.2. BFS	5
4.3. BFS Grid (4 y 8 direcciones)	6
4.4. DFS	6
4.5. DFS Grid (4 y 8 direcciones)	6
4.6. Dijkstra	7
4.7. Floyd Warshall	7
4.8. Hopcroft-Karp (matching bipartito)	7
4.9. Kuhn (matching bipartito)	8
4.10. Kruskal	8
4.11. Lowest Common Ancestor (LCA - Binary Lifting)	8
4.12. Tarjan (SCC)	8
4.13. Topological Sort	9

5. Matematicas	9
5.1. BinPow y modular inverse	9
5.2. Catalan (conteo de BST)	9
5.3. CRT (Chinese Remainder Theorem)	9
5.4. Extended Euclid	10
5.5. Pascal (coeficiente binomial)	10
5.6. Reducir fracciones	10
5.7. Shifts y rotaciones bit a bit	10
5.8. Criba de Eratostenes y factorizacion con SPF	10
6. Strings	11
6.1. Hashing	11
6.2. KMP	11
6.3. Trie	11
7. Probabilidad y Combinatoria	12
7.1. Permutaciones y Variaciones	12
7.2. Combinaciones (Coeficientes Binomiales)	12
7.3. Combinaciones con Repetición (Stars and Bars / Bolas y Barras)	12
7.4. Desarreglos (Derangements / !n)	12
7.5. Principio de Inclusión-Exclusión (PIE)	12
7.6. Probabilidad B'asica	12
7.7. Probabilidad Condicional	13
7.8. Probabilidad Total	13
7.9. Teorema de Bayes	13
7.10. Probabilidad en Varios Pasos	13
7.11. "Al Menos", "Exactamente" y "A lo M'as"	13
7.12. Combinaciones y Probabilidad	13
7.13. Distribuci'on Binomial	14
7.14. Distribuci'on Geom'etrica	14
7.15. Distribuci'on Geom'etrica: "Al Menos"	14
7.16. Valor Esperado	14
7.17. Linealidad de la Esperanza	14
8. template	15

1. Búsqueda

1.1. Búsqueda binaria

```

1 // Búsqueda binaria
2 int l = -1, r = arr.size(), ans = -1;
3 while (r - l > 1) {
4     int mid = l + (r - l) / 2;
5     if (arr[mid] >= x) {
6         r = mid;
7     } else {
8         l = mid;
9     }
10 }

```

1.2. Búsqueda ternaria

```

1 // Búsqueda ternaria
2 int l = -1, r = n;
3 while (r - l > 2) {
4     int m1 = l + (r - l) / 3;
5     int m2 = r - (r - l) / 3;
6
7     if (arr[m1] < arr[m2]) {
8         l = m1;
9     } else {
10        r = m2;
11    }
12 }
13 int best = l + 1;
14 for (int i = l + 1; i < r; i++) {
15     if (arr[i] > arr[best]) best = i;
16 }

```

2. Estructuras de datos

2.1. Fenwick Tree (BIT)

```

1 const int N = 2e5 + 5;
2 int T[N];
3
4 int query(int i){
5     int r = 0;
6     while(i > 0){
7         r += T[i];
8         i -= i & -i;
9     }
10    return r;
11 }
12
13 void add(int i, int val){
14     while(i < N){
15         T[i] += val;
16         i += i & -i;
17     }
18 }
19

```

2.2. Segment Tree (min)

```

1 const int N = 2e5 + 5;
2 int T[4*N];
3 int arr[N];
4 int n;
5
6 void init(int i, int tl, int tr){
7     if(tl == tr){
8         T[i] = arr[tl];
9         return;
10    }
11    int tm = (tl + tr) / 2;
12    init(2 * i + 1, tl, tm);
13    init(2 * i + 2, tm + 1, tr);
14
15    T[i] = min(T[2 * i + 1], T[2 * i + 2]);
16 }
17
18 void update(int i, int tl, int tr, int idx, int val){
19     if(tl == tr){
20         arr[idx] = val;
21         T[i] = val;
22         return;
23     }
24     int tm = (tl + tr) / 2;
25     if(idx <= tm){
26         update(2*i+1, tl, tm, idx, val);
27     }else{
28         update(2*i+2, tm+1, tr, idx, val);
29     }
30     T[i] = min(T[2*i+1], T[2*i+2]);
31 }
32
33 int query(int i, int tl, int tr, int l, int r){
34     if(l <= tl && tr <= r){
35         return T[i];
36     }
37     if(r < tl || tr < l){
38         return 1e9;
39     }
40     int tm = (tl + tr) / 2;
41
42     int p1 = query(2*i+1, tl, tm, l, r);
43     int p2 = query(2*i+2, tm+1, tr, l, r);
44
45     return min(p1, p2);
46 }
47
48 int query(int l, int r){
49     return query(0, 0, n-1, l, r);
50 }
51 void update(int idx, int val){
52     return update(0, 0, n-1, idx, val);
53 }

```

2.3. Segment Tree con Lazy Propagation (min + rango suma)

```

1 const int N = 2e5 + 5;
2 const int INF = 1e9;
3 int T[4*N];
4 int lazy[4*N];

```

```

5 int arr[N];
6 int n;
7
8 void push(int i, int tl, int tr){
9     if(lazy[i] != 0){
10         T[i] += lazy[i];
11
12         if(tl != tr){
13             lazy[2*i+1] += lazy[i];
14             lazy[2*i+2] += lazy[i];
15         }
16         lazy[i] = 0;
17     }
18 }
19
20 void init(int i, int tl, int tr){
21     if(tl == tr){
22         T[i] = arr[tl];
23         return;
24     }
25     int tm = (tl + tr) / 2;
26     init(2 * i + 1, tl, tm);
27     init(2 * i + 2, tm + 1, tr);
28
29     T[i] = min(T[2 * i + 1], T[2 * i + 2]);
30 }
31
32 void update(int i, int tl, int tr, int l, int r, int val){
33     push(i, tl, tr);
34
35     if(l > r) return;
36
37     if(tl == l && tr == r){
38         lazy[i] += val;
39         push(i, tl, tr);
40         return;
41     }
42     int tm = (tl + tr) / 2;
43     update(2*i+1, tl, tm, l, min(r,tm), val);
44     update(2*i+2, tm+1, tr, max(l,tm+1), r, val);
45
46     T[i] = min(T[2*i+1], T[2*i+2]);
47 }
48
49 int query(int i, int tl, int tr, int l, int r){
50     push(i, tl, tr);
51     if(l > r) return INF;
52
53     if(tl == l && tr == r){
54         return T[i];
55     }
56     int tm = (tl + tr) / 2;
57
58     int p1 = query(2*i+1, tl, tm, l, min(r,tm));
59     int p2 = query(2*i+2, tm+1, tr, max(l,tm+1), r);
60
61     return min(p1,p2);
62 }
63
64 int query(int l, int r){
65     return query(0,0,n-1,l,r);
66 }
67
68 void update(int l, int r, int val){
69     return update(0,0,n-1,l,r,val);

```

```
70 }
```

2.4. Segment Tree Persistente (min point update)

```

1 const int N = 2e5 + 5;
2 const int INF = 1e9;
3
4 struct Node {
5     int val;
6     int l, r;
7 };
8
9 Node tree[N * 40]; // 40 * N para O(log N) por modificacion
10 int roots[N];
11 int nodes_cnt = 0;
12
13 int init(int tl, int tr, int arr[]) {
14     int id = ++nodes_cnt;
15     if (tl == tr) {
16         tree[id].val = arr[tl];
17         return id;
18     }
19     int tm = (tl + tr) / 2;
20     tree[id].l = init(tl, tm, arr);
21     tree[id].r = init(tm + 1, tr, arr);
22     tree[id].val = min(tree[tree[id].l].val, tree[tree[id].r].val);
23     return id;
24 }
25
26 int update(int prev_id, int tl, int tr, int idx, int val) {
27     int id = ++nodes_cnt;
28     tree[id] = tree[prev_id];
29     if (tl == tr) {
30         tree[id].val = val;
31         return id;
32     }
33     int tm = (tl + tr) / 2;
34     if (idx <= tm)
35         tree[id].l = update(tree[prev_id].l, tl, tm, idx, val);
36     else
37         tree[id].r = update(tree[prev_id].r, tm + 1, tr, idx, val);
38
39     tree[id].val = min(tree[tree[id].l].val, tree[tree[id].r].val);
40     return id;
41 }
42
43 int query(int id, int tl, int tr, int l, int r) {
44     if (l > r || !id) return INF;
45     if (tl == l && tr == r) return tree[id].val;
46     int tm = (tl + tr) / 2;
47     return min(query(tree[id].l, tl, tm, l, min(r, tm)),
48               query(tree[id].r, tm + 1, tr, max(l, tm + 1), r));
49 }
50
51 // Uso basico:
52 // roots[0] = build(0, n-1, arr);
53 // roots[i] = update(roots[0], 0, n-1, idx, val);
54 // int ans = query(roots[v], 0, n-1, l, r);

```

2.5. UnionFind (DSU)

```
1 struct DSU{
2     vector<int> parent, size;
3     DSU(int n) {
4         parent.resize(n + 1);
5         size.resize(n + 1, 1);
6         for (int i = 1; i <= n; i++)
7             parent[i] = i;
8     }
9     int find(int v){
10        if(v == parent[v])
11            return v;
12        return parent[v] = find(parent[v]);
13    }
14
15    void unionSet(int a, int b) {
16        a = find(a);
17        b = find(b);
18        if (a != b) {
19            if (size[a] < size[b])
20                swap(a, b);
21            parent[b] = a;
22            size[a] += size[b];
23        }
24    }
25};
```

2.6. Sparse Table (RMQ $O(1)$)

```
1 const int N = 2e5 + 5, LOGN = 18;
2 int st[LOGN][N], lg[N];
3
4 void build(const vector<int>& a) {
5     int n = a.size();
6     lg[1] = 0;
7     for (int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;
8     for (int i = 0; i < n; i++) st[0][i] = a[i];
9     for (int j = 1; j < LOGN; j++)
10         for (int i = 0; i + (1 << j) <= n; i++)
11             st[j][i] = min(st[j-1][i], st[j-1][i + (1 << (j-1))]);
12 }
13
14 int query(int l, int r) { // Rango [l, r]
15     int j = lg[r - l + 1];
16     return min(st[j][l], st[j][r - (1 << j) + 1]);
17 }
```

2.7. Ordered Set (PBDS)

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 template<typename T>
6 using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
7     tree_order_statistics_node_update>;
```

```
8 // Uso basico:
9 // ordered_set<int> st;
10 // st.insert(x);
11 // st.order_of_key(x); // Cantidad de elementos estrictamente menores a x
12 // *st.find_by_order(k); // Iterador al k-esimo elemento (0-indexed)
```

3. Geometria

3.1. Graham's Scan

```
1 struct Point{
2     int x, y;
3     bool operator<(const Point &other) const {
4         if (x != other.x) return x < other.x;
5         return y < other.y;
6     }
7
8     bool operator==(const Point &other) const {
9         return x == other.x && y == other.y;
10    }
11};
12
13 void printPoints(const vector<Point>& points) {
14     for (const auto& point : points) {
15         cout << "(" << point.x << ", " << point.y << ")" << endl;
16     }
17 }
18
19 /*
20  *Function that returns
21  *1 if the points are clockwise,
22  *-1 if the points are anticlockwise,
23  *0 if they are colinear
24  */
25 int areClockwise(Point p1, Point p2, Point p3){
26     int ax = p1.x, ay = p1.y;
27     int bx = p2.x, by = p2.y;
28     int cx = p3.x, cy = p3.y;
29
30     int result = by*(cx - ax) + ay*(bx - cx) + cy*(ax - bx);
31     return result > 0 ? 1 : result < 0 ? -1 : 0;
32 }
33
34 vector<Point> grahamsScan(vector<Point> points){
35     sort(points.begin(), points.end());
36     int n = points.size();
37
38     vector<Point> up, down;
39
40     Point firstPoint = points[0], lastPoint = points[n-1];
41
42     down.push_back(firstPoint);
43     up.push_back(firstPoint);
44
45     for (int i = 1; i < n; i++)
46     {
47         Point currentPoint = points[i];
48         int clockwise = areClockwise(firstPoint, currentPoint, lastPoint);
49         if (i == n-1 || clockwise > 0) {
50             while (up.size() >= 2 && areClockwise(up[up.size() - 2],
51                 up[up.size()-1], currentPoint) < 1) {
```

```

51         up.pop_back();
52     }
53     up.push_back(currentPoint);
54 }
55
56     if (i == n-1 || clockwise < 0) {
57         while (down.size() >= 2 && areClockwise(down[down.size() - 2],
58             down[down.size()-1], currentPoint) > -1) {
59             down.pop_back();
60         }
61         down.push_back(currentPoint);
62     }
63 }
64
65 points.clear();
66 for (int i = 0; i < up.size(); i++) {
67     points.push_back(up[i]);
68 }
69 for (int i = 0; i < down.size(); i++) {
70     points.push_back(down[i]);
71 }
72
73 sort(points.begin(), points.end());
74
75 points.resize(unique(points.begin(), points.end()) - points.begin());
76
77 return points;
78 }

```

4. Grafos

4.1. Bellman Ford

```

1 vector<int> bellmanFord(int n, vector<vector<pair<int,int>>> &G, int src) {
2     vector<int> dist(n, INF);
3     dist[src] = 0;
4
5     for (int i = 0; i < n - 1; i++) {
6         for (int u = 0; u < n; u++) {
7             if (dist[u] == INF) continue;
8             for (auto [v, w] : G[u]) {
9                 if (dist[u] + w < dist[v]) {
10                     dist[v] = dist[u] + w;
11                 }
12             }
13         }
14     }
15
16     for (int u = 0; u < n; u++) {
17         if (dist[u] == INF) continue;
18         for (auto [v, w] : G[u]) {
19             if (dist[u] + w < dist[v]) {
20                 return {};
21             }
22         }
23     }
24
25     return dist;
26 }

```

4.2. BFS

```

1 vector<int> bfs(vector<vector<int>>& g , int u) {
2     vector<int> dis(g.size(), -1);
3     queue<int> q;
4     dis[u] = 0;
5     q.push(u);
6     while (!q.empty()) {
7         int node = q.front();
8         q.pop();
9         for (int v : g[node]) {
10             if (dis[v] == -1) {
11                 dis[v] = dis[node] + 1;
12                 q.push(v);
13             }
14         }
15     }
16     return dis;
17 }
18
19 vector<int> bfs(vector<vector<int>>& g , int u,int t) {
20     vector<int> dis(g.size(), -1);
21     vector<int> parent(g.size(), -1);
22     queue<int> q;
23     dis[u] = 0;
24     q.push(u);
25     while (!q.empty()) {
26         int node = q.front();
27         q.pop();
28         for (int v : g[node]) {
29             if (dis[v] == -1) {
30                 dis[v] = dis[node] + 1;
31                 parent[v] = node;
32                 q.push(v);
33             }
34         }
35     }
36
37     if(dis[t] == -1) return {};
38     vector<int> path;
39     for(int v = t; v != -1; v = parent[v]){
40         path.push_back(v);
41     }
42     reverse(all(path));
43     return path;
44 }
45
46 int ans = INF;
47
48 void bfs(vector<vector<int>>& g , int u) {
49     vector<int> dis(g.size(), -1);
50     vector<int> parents(g.size(),-1);
51     queue<int> q;
52     dis[u] = 0;
53     q.push(u);
54     while (!q.empty()) {
55         int node = q.front();
56         q.pop();
57         for (int v : g[node]) {
58             if (dis[v] == -1) {
59                 dis[v] = dis[node] + 1;
60                 parents[v] = node;
61                 q.push(v);
62             }else if(parents[node] != v){
63                 int len = dis[node] + dis[v] + 1;

```

```

64         if(len < ans)ans = len;
65     }
66 }
67 }
68 }

```

4.3. BFS Grid (4 y 8 direcciones)

```

1 int n, m;
2 vector<vector<bool>> vis;
3 vector<string> grid;
4 int dx[4] = {1, -1, 0, 0};
5 int dy[4] = {0, 0, 1, -1};
6
7 void bfs(int sx, int sy) {
8     queue<pair<int,int>> q;
9     q.push({sx, sy});
10    vis[sx][sy] = true;
11
12    while (!q.empty()) {
13        auto [x, y] = q.front(); q.pop();
14        for (int k = 0; k < 4; k++) {
15            int nx = x + dx[k];
16            int ny = y + dy[k];
17            if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
18                !vis[nx][ny] && grid[nx][ny] != '#') {
19                vis[nx][ny] = true;
20                q.push({nx, ny});
21            }
22        }
23    }
24 }
25
26 int dx8[8] = {1, -1, 0, 0, 1, 1, -1, -1};
27 int dy8[8] = {0, 0, 1, -1, 1, -1, 1, -1};
28
29 void bfs8(int sx, int sy) {
30     queue<pair<int,int>> q;
31     q.push({sx, sy});
32     vis[sx][sy] = true;
33
34     while (!q.empty()) {
35         auto [x, y] = q.front(); q.pop();
36         for (int k = 0; k < 8; k++) {
37             int nx = x + dx8[k];
38             int ny = y + dy8[k];
39             if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
40                 !vis[nx][ny] && grid[nx][ny] != '#') {
41                 vis[nx][ny] = true;
42                 q.push({nx, ny});
43             }
44         }
45     }
46 }

```

4.4. DFS

```

1 void dfs(int v, vector<bool>& visited, const vector<vector<int>>& graph) {
2     visited[v] = true;

```

```

3     for (int u : graph[v]) {
4         if (!visited[u]) {
5             dfs(u, visited, graph);
6         }
7     }
8 }
9
10 bool dfsCiclo(int v,int parent, vector<bool>& visi, vector<vector<int>>& gr){
11     visi[v] = true;
12     for(int vis : gr[v]){
13         if(!visi[vis]){
14             if(dfsCiclo(vis, v,visi, gr))
15                 return true;
16         }else if(vis != parent){
17             return true;
18         }
19     }
20     return false;
21 }
22
23 void tra(int n,vector<vii> G){
24     vector<vector<pair<ll,ll>>> Trans(n + 1);
25     foree(i,1,n){
26         for(auto g : G[i]){
27             ll v = g.F;
28             ll cost = g.S;
29             Trans[v].pb({i,cost});
30         }
31     }
32 }

```

4.5. DFS Grid (4 y 8 direcciones)

```

1 int n, m;
2 vector<vector<bool>> vis;
3 vector<string> grid;
4 int dx[4] = {1, -1, 0, 0};
5 int dy[4] = {0, 0, 1, -1};
6
7 int dx8[8] = {1, -1, 0, 0, 1, 1, -1, -1};
8 int dy8[8] = {0, 0, 1, -1, 1, -1, 1, -1};
9
10 void dfs(int x, int y) {
11     vis[x][y] = true;
12     for (int k = 0; k < 4; k++) {
13         int nx = x + dx[k];
14         int ny = y + dy[k];
15         if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
16             !vis[nx][ny] && grid[nx][ny] != '#') {
17             dfs(nx, ny);
18         }
19     }
20 }
21
22 void dfs8(int x, int y) {
23     vis[x][y] = true;
24     for (int k = 0; k < 8; k++) {
25         int nx = x + dx8[k];
26         int ny = y + dy8[k];
27         if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
28             !vis[nx][ny] && grid[nx][ny] != '#') {
29             dfs8(nx, ny);

```

```

30     }
31 }
32 }

```

4.6. Dijkstra

```

1 vector<ll> dijkstra(vector<vector<pair<ll,ll>>> G, int n, int s){
2     vector<ll> dis(n+1, INF);
3     dis[s]=0;
4     priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<pair<ll,ll>>> pq;
5     pq.push({0,s});
6     while(!pq.empty()){
7         pair<ll,ll> menor = pq.top();
8         pq.pop();
9         int w = menor.first;
10        int nodo = menor.second;
11        if(dis[nodo] < w) continue;
12        for( auto to : G[nodo]){
13            int v = to.first;
14            int cost = to.second;
15            if(dis[nodo]+cost < dis[v]){
16                dis[v] = dis[nodo] + cost;
17                pq.push({dis[v],v});
18            }
19        }
20    }
21    return dis;
22 }

```

4.7. Floyd Warshall

```

1 vector<vector<ll>> floydWarshall(vector<vector<pair<ll,ll>>> G, int n){
2     vector<vector<ll>> dis(n+1, vector<ll>(n+1,INF));
3     for(int i = 1; i <= n; i++){
4         dis[i][i] = 0;
5     }
6     for(int u = 1; u <= n; ++u){
7         for(auto to : G[u]){
8             ll v = to.first, w = to.second;
9             dis[u][v] = min(dis[u][v],w);
10            dis[v][u] = min(dis[v][u],w);
11        }
12    }
13    for(int k = 1; k <= n; ++k){
14        for(int u = 1; u <= n; ++u){
15            for(int v = 1; v <= n; ++v){
16                dis[u][v] = min(dis[u][v],dis[u][k]+dis[k][v]);
17            }
18        }
19    }
20    return dis;
21 }

```

4.8. Hopcroft-Karp (matching bipartito)

```

1 const int INF = 1e9;
2 struct HopcroftKarp {
3     int n, m;
4     vector<vector<int>> adj;
5     vector<int> matchU, matchV, dist;
6
7     HopcroftKarp(int n, int m) : n(n), m(m), adj(n + 1), matchU(n + 1, 0), matchV(m
8         + 1, 0), dist(n + 1) {}
9
10    void add_edge(int u, int v) {
11        adj[u].push_back(v);
12    }
13
14    bool bfs() {
15        queue<int> q;
16        for (int u = 1; u <= n; u++) {
17            if (matchU[u] == 0) {
18                dist[u] = 0;
19                q.push(u);
20            } else {
21                dist[u] = INF;
22            }
23        }
24        dist[0] = INF;
25
26        while (!q.empty()) {
27            int u = q.front();
28            q.pop();
29
30            if (dist[u] < dist[0]) {
31                for (int v : adj[u]) {
32                    if (dist[matchV[v]] == INF) {
33                        dist[matchV[v]] = dist[u] + 1;
34                        q.push(matchV[v]);
35                    }
36                }
37            }
38        }
39        return dist[0] != INF;
40    }
41
42    bool dfs(int u) {
43        if (u != 0) {
44            for (int v : adj[u]) {
45                if (dist[matchV[v]] == dist[u] + 1) {
46                    if (dfs(matchV[v])) {
47                        matchV[v] = u;
48                        matchU[u] = v;
49                        return true;
50                    }
51                }
52            }
53            dist[u] = INF;
54            return false;
55        }
56        return true;
57    }
58
59    int max_matching() {
60        int matching = 0;
61        while (bfs()) {
62            for (int u = 1; u <= n; u++) {
63                if (matchU[u] == 0 && dfs(u)) {
64                    matching++;
65                }
66            }
67        }
68    }
69 }

```

```

65     }
66 }
67     return matching; // Retorna el numero maximo de matches
68 }
69 };

```

4.9. Kuhn (matching bipartito)

```

1 vector<vector<int>> g;
2 vector<int> mt;
3 vector<bool> used;
4
5 bool try_kuhn(int v) {
6     if (used[v]) return false;
7     used[v] = true;
8     for (int to : g[v]) {
9         if (mt[to] == -1 || try_kuhn(mt[to])) {
10             mt[to] = v;
11             return true;
12         }
13     }
14     return false;
15 }
16
17 int kuhn(int n1) {
18     int match = 0;
19     for (int v = 0; v < n1; v++) {
20         used.assign(n1, false);
21         if (try_kuhn(v))
22             match++;
23     }
24     return match;
25 }

```

4.10. Kruskal

```

1 ll kruskal(vector<pair<ll,pair<int,int>>> edges, int n){
2     sort(edges.begin(), edges.end());
3     DSU dsu(n);
4     int cntAristas = 0;
5     ll res = 0;
6     for(auto e : edges){
7         ll peso = e.first;
8         int u = e.S.first;
9         int v = e.S.second;
10        if(dsu.unionSet(u, v)){
11            cntAristas++;
12            res += peso;
13        }
14        if(cntAristas == n-1){
15            return res;
16        }
17    }
18    if(cntAristas < n - 1){
19        return -1;
20    }
21    return res;
22 }

```

4.11. Lowest Common Ancestor (LCA - Binary Lifting)

```

1 const int N = 2e5 + 5, LOGN = 20;
2 vector<int> adj[N];
3 int up[N][LOGN], depth[N];
4
5 void dfs(int u, int p = 0, int d = 0) {
6     depth[u] = d;
7     up[u][0] = p;
8     for (int i = 1; i < LOGN; i++)
9         up[u][i] = up[up[u][i-1]][i-1];
10    for (int v : adj[u])
11        if (v != p) dfs(v, u, d + 1);
12 }
13
14 int lca(int u, int v) {
15     if (depth[u] < depth[v]) swap(u, v);
16     for (int i = LOGN - 1; i >= 0; i--) {
17         if (depth[u] - (1 << i) >= depth[v]) {
18             u = up[u][i];
19         }
20     }
21     if (u == v) return u;
22     for (int i = LOGN - 1; i >= 0; i--) {
23         if (up[u][i] != up[v][i]) {
24             u = up[u][i];
25             v = up[v][i];
26         }
27     }
28     return up[u][0];
29 }

```

4.12. Tarjan (SCC)

```

1 const int N = 1e5+5;
2
3 vector<int> G[N];
4 int n, m;
5
6 bool onStack[N], visited[N];
7 int ids[N], low[N], cnt = 1, sccCnt = 0;
8 stack<int> st;
9 int sccs[N];
10
11 void dfs(int i) {
12     st.push(i);
13     onStack[i] = 1;
14     ids[i] = low[i] = cnt++;
15
16     for (int to : G[i]) {
17         if (!visited[to]) {
18             visited[to] = 1;
19             dfs(to);
20         }
21         if (onStack[to]) {
22             low[i] = min(low[i], low[to]);
23         }
24     }
25
26     if (ids[i] == low[i]) {
27         sccCnt++;
28         while (1) {

```

```

29         int node = st.top(); st.pop();
30         onStack[node] = 0;
31         sccs[node] = sccCnt;
32         if (node == i) break;
33     }
34 }
35 }
36
37 void findSCC() {
38     for (int i=1; i<=n; ++i) {
39         if (visited[i]) continue;
40         visited[i] = 1;
41         dfs(i);
42     }
43 }

```

4.13. Topological Sort

```

1 vector<int> topologicalSort(int n, vector<vector<int>>> &G) {
2     vector<int> indeg(n, 0), topo;
3     queue<int> q;
4
5     for (int u = 0; u < n; u++) {
6         for (int v : G[u]) {
7             indeg[v]++;
8         }
9     }
10
11     for (int i = 0; i < n; i++) {
12         if (indeg[i] == 0) q.push(i);
13     }
14
15     while (!q.empty()) {
16         int u = q.front(); q.pop();
17         topo.push_back(u);
18
19         for (int v : G[u]) {
20             indeg[v]--;
21             if (indeg[v] == 0) {
22                 q.push(v);
23             }
24         }
25     }
26
27     if ((int)topo.size() != n) {
28         return {};
29     }
30
31     return topo;
32 }

```

5. Matematicas

5.1. BinPow y modular inverse

```

1 ll binpow(ll a, ll b, ll m) {
2     a %= m;

```

```

3     ll res = 1;
4     while (b > 0) {
5         if (b & 1)
6             res = res * a % m;
7         a = a * a % m;
8         b >>= 1;
9     }
10    return res;
11 }
12 ll modInverse(ll a, ll p) {
13     return binpow(a, p - 2, p);
14 }

```

5.2. Catalan (conteo de BST)

```

1 int countBST(int n){
2     vector<int> dp(n+1,0);
3     dp[0] = dp[1] = 1;
4
5     for(int i = 2; i<=n; i++){
6         for(int j=1 ; j<=i;j++){
7             dp[i] += (dp[j-1] * dp[i-j]);
8         }
9     }
10    return dp[n];
11 }

```

5.3. CRT (Chinese Remainder Theorem)

```

1 constexpr long long safe_mod(long long x, long long m) {
2     x %= m;
3     if (x < 0) x += m;
4     return x;
5 }
6
7 constexpr std::pair<long long, long long> inv_gcd(long long a, long long b) {
8     a = safe_mod(a, b);
9     if (a == 0) return {b, 0};
10    long long s = b, t = a;
11    long long m0 = 0, m1 = 1;
12    while (t) {
13        long long u = s / t;
14        s -= t * u;
15        m0 -= m1 * u;
16        auto tmp = s;
17        s = t;
18        t = tmp;
19        tmp = m0;
20        m0 = m1;
21        m1 = tmp;
22    }
23    if (m0 < 0) m0 += b / s;
24    return {s, m0};
25 }
26
27 std::pair<long long, long long> crt(const std::vector<long long>& r,
28                                   const std::vector<long long>& m) {
29     assert(r.size() == m.size());
30     int n = int(r.size());

```

```

31 long long r0 = 0, m0 = 1;
32 for (int i = 0; i < n; i++) {
33     assert(1 <= m[i]);
34     long long r1 = safe_mod(r[i], m[i]), m1 = m[i];
35     if (m0 < m1) {
36         std::swap(r0, r1);
37         std::swap(m0, m1);
38     }
39     if (m0 % m1 == 0) {
40         if (r0 % m1 != r1) return {0, 0};
41         continue;
42     }
43     long long g, im;
44     std::tie(g, im) = inv_gcd(m0, m1);
45     long long u1 = (m1 / g);
46     if ((r1 - r0) % g) return {0, 0};
47     long long x = (r1 - r0) / g % u1 * im % u1;
48     r0 += x * m0;
49     m0 *= u1;
50     if (r0 < 0) r0 += m0;
51 }
52 return {r0, m0};
53 }

```

5.4. Extended Euclid

```

1 int gcd(int a, int b, int& x, int& y) {
2     if (b == 0) {
3         x = 1;
4         y = 0;
5         return a;
6     }
7     int x1, y1;
8     int d = gcd(b, a % b, x1, y1);
9     x = y1;
10    y = x1 - y1 * (a / b);
11    return d;
12 }

```

5.5. Pascal (coeficiente binomial)

```

1 int binomialCoefficient(int n, int k){
2     vector<vector<int>> table(n + 1, vector<int>(k+1,0));
3     for(int i = 0; i <= n; i++){
4         for(int j = 0; j <= min(i,k); j++){
5             if(j==0 || j == i){
6                 table[i][j] = 1;
7             }else{
8                 table[i][j] = table[i-1][j-1] + table[i-1][j];
9             }
10        }
11    }
12    return table[n][k];
13 }

```

5.6. Reducir fracciones

```

1 void reduceFraction(int a, int b, int& n, int& d) {
2     int g = gcd(a, b);
3     n = a / g;
4     d = b / g;
5 }

```

5.7. Shifts y rotaciones bit a bit

```

1 inline long long shiftLeft(long long num, int k, int n) {
2     k %= n;
3     return (num << k) & ((1LL << n) - 1);
4 }
5
6 inline long long shiftRight(long long num, int k, int n) {
7     k %= n;
8     return (num >> k) & ((1LL << n) - 1);
9 }
10
11 inline long long rotateLeft(long long num, int k, int n) {
12     k %= n;
13     return ((num << k) | (num >> (n - k))) & ((1LL << n) - 1);
14 }
15
16 inline long long rotateRight(long long num, int k, int n) {
17     k %= n;
18     return ((num >> k) | (num << (n - k))) & ((1LL << n) - 1);
19 }

```

5.8. Criba de Eratostenes y factorizacion con SPF

```

1 vector<bool> sieve(int n){
2     vector<bool> primes(n + 1, true);
3     primes[0] = primes[1] = false;
4
5     for(int i = 2; i * i <= n; i++){
6         if(primes[i])
7             for(int j = i * i; j <= n; j+=i)
8                 primes[j] = false;
9     }
10    return primes;
11 }
12
13 vector<int> sieve(int n) {
14     vector<int> spf(n + 1);
15     for (int i = 1; i <= n; ++i) {
16         spf[i] = i;
17     }
18     for (int i = 2; i * i <= n; ++i) {
19         if (spf[i] == i) {
20             for (int j = i * i; j <= n; j += i) {
21                 if (spf[j] == j) {
22                     spf[j] = i;
23                 }
24             }
25         }
26     }
27 }

```

```

27     return spf;
28 }
29
30 vector<int> facPrime(int n, vector<int>& spf) {
31     vector<int> fac;
32     while (n > 1) {
33         int p = spf[n];
34         fac.push_back(p);
35         while (n % p == 0)
36             n /= p;
37     }
38     return fac;
39 }
40
41 vi numDivs(int n) {
42     vi divs;
43     for(int i = 1; i * i <= n; i++) {
44         if(n % i == 0) {
45             divs.pb(i);
46             if(i != n / i) divs.pb(n / i);
47         }
48     }
49     sort(all(divs));
50     return divs;
51 }

```

6. Strings

6.1. Hashing

```

1 #define bint __int128
2 struct Hash {
3     bint MOD=212345678987654321LL,P=1777771,PI=106955741089659571LL;
4     vector<bint> h,pi;
5     Hash(string& s){
6         assert((P*PI)%MOD==1);
7         h.resize(s.size()+1); pi.resize(s.size()+1);
8         h[0]=0; pi[0]=1;
9         bint p=1;
10        for(int i = 1; i < s.size()+1; i++){
11            h[i] = (h[i-1] + p * s[i-1]) % MOD;
12            pi[i] = (pi[i-1] * PI) % MOD;
13            p = (p * P) % MOD;
14        }
15    }
16    ll get(int s, int e){
17        return ((h[e]-h[s]+MOD)%MOD) * pi[s] % MOD;
18    }
19 };

```

6.2. KMP

```

1 vector<int> prefix_function(string s) {
2     int n = (int)s.length();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i-1];

```

```

6         while (j > 0 && s[i] != s[j])
7             j = pi[j-1];
8         if (s[i] == s[j]) j++;
9         pi[i] = j;
10    }
11    return pi;
12 }
13
14 vector<int> kmp_search(const string& text, const string& pattern) {
15     int n = text.size(), m = pattern.size();
16     vector<int> pi = prefix_function(pattern);
17     vector<int> positions;
18
19     int j = 0;
20     for (int i = 0; i < n; i++) {
21         while (j > 0 && text[i] != pattern[j])
22             j = pi[j - 1];
23         if (text[i] == pattern[j]) j++;
24         if (j == m) {
25             positions.push_back(i - m + 1);
26             j = pi[j - 1];
27         }
28     }
29     return positions;
30 }

```

6.3. Trie

```

1 struct Node {
2     Node *next[26];
3     bool overNode;
4
5     Node(){
6         overNode = false;
7         for(int i = 0; i < 26; i++){
8             next[i] = NULL;
9         }
10    }
11
12    void insertWord(string s){
13        Node *cur = this;
14        int size = s.length();
15        for(int i = 0; i < size; i++){
16            int imap = s[i] - 'a';
17            if(!cur->next[imap]){
18                cur->next[imap] = new Node();
19            }
20            cur = cur->next[imap];
21        }
22        cur->overNode = true;
23    }
24
25    bool searchWord(string s){
26        Node *cur = this;
27        int size = s.length();
28        bool founded = true;
29        for(int i = 0; i < size; i++){
30            int imap = s[i] - 'a';
31            if(!cur->next[imap]){
32                founded = false;
33                break;
34            }

```

```

35     cur = cur->next[imap];
36 }
37 return founded && cur->overNode;
38 }
39 };

```

7. Probabilidad y Combinatoria

7.1. Permutaciones y Variaciones

- Permutación simple (ordenar n elementos distintos):

$$P_n = n!$$

- Permutación con repetición (multiconjunto con $k_1, k_2, \dots, k_r$ repetidos):

$$P_n^{k_1, k_2, \dots, k_r} = \frac{n!}{k_1! k_2! \dots k_r!} \quad \text{donde } \sum k_i = n$$

- Permutación circular (ordenar n elementos en un círculo):

$$P_{\text{circ}}(n) = (n-1)!$$

- Variaciones sin repetición (elegir y ordenar k elementos de n):

$$P(n, k) = V(n, k) = \frac{n!}{(n-k)!} = n \cdot (n-1) \cdots (n-k+1)$$

- Variaciones con repetición (elegir y ordenar k elementos permitiendo repetir):

$$VR(n, k) = n^k$$

7.2. Combinaciones (Coeficientes Binomiales)

Seleccionar k elementos de n sin importar el orden:

$$\binom{n}{k} = C(n, k) = \frac{n!}{k!(n-k)!}$$

Propiedades clave para ICPC:

- Simetría: $\binom{n}{k} = \binom{n}{n-k}$
- Identidad de Pascal: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$
- Suma de la fila: $\sum_{k=0}^n \binom{n}{k} = 2^n$
- Identidad del palo de Hockey (Hockey-stick): $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$
- Identidad de Vandermonde: $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$

7.3. Combinaciones con Repetición (Stars and Bars / Bolas y Barras)

Formas de distribuir k objetos indistinguibles en n recipientes distinguibles:

$$CR(n, k) = \binom{n+k-1}{k} = \binom{n+k-1}{n-1}$$

- Soluciones enteras no negativas ($x_1 + x_2 + \dots + x_n = k$ con $x_i \geq 0$): $\binom{n+k-1}{k}$
- Soluciones enteras estrictamente positivas ($x_i \geq 1$ con $k \geq n$): $\binom{k-1}{n-1}$

7.4. Desarrreglos (Derangements / !n)

Permutaciones de n elementos donde ningún elemento queda en su posición original:

$$!n = D_n = (n-1)(D_{n-1} + D_{n-2}) = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

Recurrencia alternativa: $D_n = nD_{n-1} + (-1)^n$ con $D_0 = 1, D_1 = 0$.

7.5. Principio de Inclusión-Exclusión (PIE)

Para n conjuntos $A_1, A_2, \dots, A_n$:

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{i=1}^n |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| - \dots + (-1)^{n-1} |A_1 \cap \dots \cap A_n|$$

7.6. Probabilidad B'asica

La probabilidad normalmente se puede pensar como:

- Casos favorables / casos posibles:

$$P(A) = \frac{\text{casos donde ocurre } A}{\text{casos posibles}}$$

- Complemento: muchas veces es m'as f'acil calcular que algo **no ocurra**:

$$P(A) = 1 - P(\text{no ocurre } A)$$

- Unión (A o B):

$$P(A \text{ o } B) = P(A) + P(B) - P(A \text{ y } B)$$

Si A y B no pueden ocurrir al mismo tiempo:

$$P(A \text{ o } B) = P(A) + P(B)$$

■ **Intersección (A y B):**

$$P(A \text{ y } B) = P(A) \cdot P(B | A)$$

Si son independientes:

$$P(A \text{ y } B) = P(A) \cdot P(B)$$

Truco ICPC: si el problema pregunta “al menos uno”, normalmente conviene pensar en el complemento:

$$P(\text{al menos uno}) = 1 - P(\text{ninguno})$$

7.7. Probabilidad Condicional

La probabilidad condicional significa:

“Sabiedo que B ocurri’o, ¿qu’e probabilidad hay de que tambi’en ocurra A?”

Se calcula como:

$$P(A | B) = \frac{P(A \text{ y } B)}{P(B)}$$

7.8. Probabilidad Total

A veces un problema tiene varios casos posibles.

Si $E_1, E_2, \dots, E_k$ son todos los casos posibles, entonces:

$$P(A) = P(E_1)P(A | E_1) + P(E_2)P(A | E_2) + \dots$$

O de forma compacta:

$$P(A) = \sum_i P(E_i)P(A | E_i)$$

Idea pr’actica:

Probabilidad de cada caso $\times$ probabilidad de que ocurra A dentro de ese caso.

Esto aparece mucho cuando un problema tiene diferentes tipos de objetos, caminos, m’aquinas, jugadores o estados.

7.9. Teorema de Bayes

Bayes sirve cuando conocemos la probabilidad de observar algo y queremos averiguar de d’onde vino.

$$P(B | A) = \frac{P(A | B)P(B)}{P(A)}$$

7.10. Probabilidad en Varios Pasos

Cuando un proceso ocurre paso por paso, normalmente se multiplican las probabilidades:

$$P(A \text{ y } B) = P(A) \cdot P(B | A)$$

Para varios pasos:

$$P(A_1, A_2, \dots, A_k) = P(A_1)P(A_2 | A_1)P(A_3 | A_1, A_2) \dots$$

7.11. “Al Menos”, “Exactamente” y “A lo M’as”

Estas expresiones aparecen constantemente en problemas.

- **Al menos k :** significa k o m’as ($P(X \geq k)$).
- **A lo más k :** significa k o menos ($P(X \leq k)$).
- **Exactamente k :** significa que ocurren exactamente k ($P(X = k)$).
- **Al menos uno:** $P(X \geq 1) = 1 - P(X = 0)$.

Truco ICPC: Si calcular “al menos uno” directamente es complicado, casi siempre conviene calcular “ninguno” y restarlo de 1.

7.12. Combinaciones y Probabilidad

Cuando todos los grupos de k elementos son igualmente probables:

$$P(A) = \frac{\text{formas favorables}}{\text{formas totales}}$$

Las combinaciones se calculan como:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Esto aparece mucho en problemas de:

- cartas;

- equipos;
- seleccionar personas;
- seleccionar elementos;
- problemas de “exactamente k ”.

7.13. Distribuci'on Binomial

Se utiliza cuando:

- hacemos exactamente N intentos;
- cada intento tiene dos resultados: 'exito o fracaso;
- la probabilidad de 'exito es siempre p ;
- los intentos son independientes.

La probabilidad de obtener exactamente k 'exitos es:

$$P(X = k) = \binom{N}{k} p^k (1 - p)^{N-k}$$

Por ejemplo:

Probabilidad de ganar exactamente 3 de 10 partidas si cada partida se gana con probabilidad 0,6.

Esperanza:

$$E[X] = N \cdot p$$

Truco: si solamente preguntan cu'antos 'exitos esperamos, no necesitas calcular toda la distribuci'on. Usa directamente: $N \cdot p$.

7.14. Distribuci'on Geom'etrica

Se utiliza cuando queremos saber:

“¿Cu'antos intentos necesito hasta conseguir el primer 'exito?”

Si cada intento tiene probabilidad p de 'exito:

$$P(X = k) = (1 - p)^{k-1} p$$

La esperanza es:

$$E[X] = \frac{1}{p}$$

Por ejemplo, si la probabilidad de ganar una partida es 0,2:

$$E[X] = \frac{1}{0,2} = 5$$

En promedio necesitamos 5 partidas para conseguir la primera victoria.

7.15. Distribuci'on Geom'etrica: “Al Menos”

Una propiedad muy 'util:

$$P(X > k) = (1 - p)^k$$

Es decir:

Para tardar m'as de k intentos, los primeros k intentos tienen que fallar.

Entonces:

$$P(X \leq k) = 1 - (1 - p)^k$$

Esto es muy frecuente en problemas donde preguntan:

“¿Cu'al es la probabilidad de conseguir al menos un 'exito en k intentos?”

La respuesta es: $1 - (1 - p)^k$.

7.16. Valor Esperado

La esperanza es simplemente el **promedio que esperamos obtener si repetimos el experimento muchas veces**.

Si X puede tomar varios valores:

$$E[X] = x_1 P(X = x_1) + x_2 P(X = x_2) + \dots = \sum_x x P(X = x)$$

Forma de pensarlo:

Multiplica cada resultado por la probabilidad de obtenerlo y suma todo.

7.17. Linealidad de la Esperanza

Esta es una de las herramientas m'as importantes para ICPC:

$$E[X + Y] = E[X] + E[Y]$$

Y tambi'en:

$$E[aX + b] = aE[X] + b$$

Lo importante es que **no necesitamos que X e Y sean independientes**.

Por ejemplo, si queremos saber cu'antos elementos cumplen cierta condici'on, podemos crear una variable indicadora para cada elemento:

$$X = X_1 + X_2 + \cdots + X_N \implies E[X] = E[X_1] + E[X_2] + \cdots + E[X_N]$$

8. template

```
1 #include <bits/stdc++.h>
2 #define int long long
3 #define FIO ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
4 #define pb push_back
5 #define endl "\n"
6 #define F first
7 #define S second
8 #define vi vector<int>
9 #define ii pair<int,int>
10 #define debug cout<<"entro"<<endl;
11 #define debug2(x) cout<<": "<<x<<endl;
12 #define vii vector<ii>
13 #define all(x) x.begin(),x.end()
14 #define allr(x) x.rbegin(),x.rend()
15 #define sz(x) (int)x.size()
16 #define double long double
17 #define forn(i,n) for(int i = 0; i < n; i++)
18 #define fore(i, b, e) for(int i = b; i < e; i++)
19 #define foree(i, b, e) for(int i = b; i <= e; i++)
20 #define forg(i, b, e, m) for(int i = b; i < e; i+=m)
21 using namespace std;
22 // const int INF = 1e9 + 7;
23 // const int tam = 1e6 + 5;
24 // int dp[tam];
25 using ll = long long;
26
27 void solve(){
28
29 }
30
31 signed main(){
32     FIO
33     // int t;
34     // cin >> t;
35     // while(t-->0)
36     solve();
37
38 }
```